A Minimalist Datatype-Generic Putback-Based Bidirectional Programming Language

Hsiang-Shang Ko[2], Tao Zan[1], and Zhenjiang Hu[1, 2]

[1] SOKENDAI (The Graduate University for Advanced Studies), Japan
{taozan,hu}@nii.ac.jp
[2] National Institute of Informatics, Japan
hsiang-shang@nii.ac.jp

1 Introduction

Theory construction in terms of Agda programming

2 Decidable partiality

The bidirectional transformations discussed in this paper are "decidably partial". By "decidable partiality" we mean that the transformations are actually total functions which wrap their results in the standard Maybe datatype:[3]

```
data Maybe (A : Set) : Set where
  just    : A → Maybe A
  nothing : Maybe A
```

The result of a piece of successful computation is wrapped in the just constructor, while failed computation is represented by nothing. The behaviour of decidably partial functions is different from the usual partial functions as formalised in terms of complete partial orders (see, e.g., Gibbons [3]): In Section 4 we will need to define programs that produce some result or fail respectively when a sub-program fails or produces some result, but this is not possible in the setting of complete partial orders (since such functions violate monotonicity, a necessary condition for computability).

We can choose to write our decidably partial bidirectional transformations directly as Maybe-programs, but when proving their well-behavedness, we will need to analyse or establish how a piece of decidably partial computation succeeds or fails, and the task would be much easier if the computation steps leading to success or failure are reified as data. (An example of such proofs is given in the definition of lens composition _↔_ in Section 3.) The reification starts from

[3] An Agda datatype is defined by listing all of its constructors along with their types in full. A datatype can be *parametrised*, as in the case of Maybe, which has a parameter A : Set, specified between the name of the datatype and the colon on the first line. (Set is the type of all (small) types.) Parameters can be referred to throughout a datatype definition.

```
runPar : {A : Set} → Par A → Maybe A
runPar (return x)  =  just x
runPar (mx >>= f) with runPar mx
runPar (mx >>= f) | just x    =  runPar (f x)
runPar (mx >>= f) | nothing  =  nothing
runPar fail  =  nothing
runPar (catch mx f my) with runPar mx
runPar (catch mx f my) | just x    =  runPar (f x)
runPar (catch mx f my) | nothing  =  runPar my
runPar (assert true then mx)  =  runPar mx
runPar (assert false then mx)  =  nothing
```

Fig. 1. Definition of *runPar*.

the following datatype Par, which is a deep embedding of the combinators that we will use for writing decidably partial programs:[4]

```
data Par : Set → Set₁ where
  return      : {A : Set} →     A → Par A
  _>>=_       : {A B : Set} → Par A → (A → Par B) → Par B
  fail        : {A : Set} →     Par A
  catch       : {A B : Set} → Par A → (A → Par B) → Par B → Par B
  assert_then_ : {A B : Set} → Bool → Par A → Par A
```

[4] Here the datatype Par is *indexed* by Set (which appears to the right of the colon on the first line), meaning that Par is in fact a Set-indexed family of simultaneously defined types.

As for the constructors, let us look at the simplest case, fail: The constructor fail constructs a value of type Par A for any A : Set. There is thus one argument to fail, namely A : Set, and the type of fail's result depends on the value of this argument (which happens to be a type). This is an example of *dependent function types*, whose result type can depend on the value of the argument. Logically, dependent function types are read as universally quantified propositions — the dependent function type $\{x : A\} \to B\ x$ is read as "for all $x : A$ the property B holds for x". (Ordinary function types, on the other hand, denote implications logically.)

Back to fail: most of the time Agda can infer what fail's argument should be based on how fail is used, so we make the argument *implicit* by enclosing it in curly brackets. (Explicit arguments, on the other hand, are enclosed in round brackets.) We can then invoke fail without supplying the argument, leaving it for Agda to infer. We de-emphasise implicit arguments typographically when they cause visual annoyance, as in this case.

An Agda name can be used as an infix (or, in general, mixfix) operator when the name contains underscores, in which case the underscores indicate where the arguments should go. For example, we can apply _>>=_ to arguments mx and f by writing either _>>=_ $mx\ f$ or mx >>= f.

```
mutual
  _↦_ : {A : Set} → Par A → A → Set₁
  return x          ↦ x′ = x ≡ x′
  mx >>= f          ↦ y  = (x : _) × (mx ↦ x) × (f x ↦ y)
  fail              ↦ x  = ⊥
  catch mx f my     ↦ y  = ((x : _) × (mx ↦ x) × (f x ↦ y)) ⊎
                           ((mx ↛) × (my ↦ y))
  assert b then mx  ↦ x  = (b ≡ true) × (mx ↦ x)
  _↛ : {A : Set} → Par A → Set₁
  return x          ↛ = ⊥
  mx >>= f          ↛ = (mx ↛) ⊎ ((x : _) × (mx ↦ x) × (f x ↛))
  fail              ↛ = ⊤
  catch mx f my     ↛ = ((mx ↛) × (my ↛)) ⊎
                        ((x : _) × (mx ↦ x) × (f x ↛))
  assert b then mx  ↛ = (b ≡ false) ⊎ ((b ≡ true) × (mx ↛))
```

Fig. 2. Definitions of $\_\mapsto\_$ and $\_\not\mapsto$.

Informally: A value of type Par A represents a piece of decidably partial computation that either successfully produces a value of type A or fails. The first two constructors return and $\_\gg\!=\_$ represent the usual monadic operators [4,7], and fail indicates failed computation. We will also need error handling, which is represented by the catch constructor. The behaviour of our catch is slightly different from the usual one: execution of catch mx f my binds the result of mx to f if mx computes successfully, or goes to execution of my if mx fails to compute. Also we will frequently need to ensure that a Bool value is true before continuing with the rest of a program, so the constructor assert_then_ is included for convenience — execution of assert b then p becomes execution of p when b is true, or fails otherwise.

Formally, the meaning of the constructors of Par is given by the interpreter $runPar : \{A : \mathsf{Set}\} \to \mathsf{Par}\ A \to \mathsf{Maybe}\ A$ shown in Figure 1. We say that $mx : \mathsf{Par}\ A$ computes to $x : A$ exactly when $runPar\ mx \equiv \mathsf{just}\ x$, and that mx fails to compute exactly when $runPar\ mx \equiv \mathsf{nothing}$. These two equalities are what we aim to reify as data next.

Our reification of decidably partial computation as the datatype Par and Agda's support of full dependent types allow us to compute for any $mx : \mathsf{Par}\ A$ and $x : A$ a type $mx \mapsto x$ whose inhabitants explain how mx computes to x step by step. The definition of this family of types is shown in Figure 2:[5] return x

[5] The type $\top$: Set has only one inhabitant, while the type $\bot$: Set has no inhabitant — logically they are read as truth and falsity respectively. The type family $\_\equiv\_ : \{A : \mathsf{Set}\} \to A \to A \to \mathsf{Set}$ is defined such that $x \equiv y$ is inhabited if and only if x and y are equal. The type former $\_\uplus\_ : \mathsf{Set} \to \mathsf{Set} \to \mathsf{Set}$ is disjoint union (disjunction), and the type former $\_\times\_ : \mathsf{Set} \to \mathsf{Set} \to \mathsf{Set}$ is cartesian product (con-

computes to x' when x is exactly x', and $mx \gg\!= f$ computes to y when there exists $x : A$ such that mx computes to x and $f\ x$ computes to y, and so on. Here our use of the term "computes to" is justified, since we can prove that

$$mx \mapsto x \qquad \text{and} \qquad runPar\ mx \equiv \mathsf{just}\ x$$

are equivalent. That is, a term of type $mx \mapsto x$ can be constructed if and only if $runPar\ mx \equiv \mathsf{just}\ x$, meaning that our reification of $runPar\ mx \equiv \mathsf{just}\ x$ as $mx \mapsto x$ is accurate. In the clause for $\mathsf{catch}\ mx\ f\ my$, a possible situation for the computation to succeed is when mx fails but my succeeds subsequently. Thus the ways a piece of computation fails have to be reified as a family of types as well. This family of types is defined as $\_\not\mapsto$ (mutually with $\_\mapsto\_$), also in Figure 2. Similarly, we can prove that

$$mx \not\mapsto \qquad \text{and} \qquad runPar\ mx \equiv \mathsf{nothing}$$

are equivalent, showing that the reification of $runPar\ mx \equiv \mathsf{nothing}$ as $mx \not\mapsto$ is again accurate. We are thus entitled to use $\_\mapsto\_$ and $\_\not\mapsto$ for stating various properties involving success or failure of decidably partial computation, in particular well-behavedness of bidirectional transformations in the next section.

3 Putback-based bidirectional transformations

We follow the classic (well-behaved) lens approach [1] to bidirectional transformations, but adopting the variation used by Pacheco et al [5]:

Definition 1 (lenses). *A (well-behaved)* lens *between a source type* $S : \mathsf{Set}$ *and a view type* $V : \mathsf{Set}$ *is a pair of functions*

$$put : S \to V \to \mathsf{Par}\ S \qquad \text{and} \qquad get : S \to \mathsf{Par}\ V$$

satisfying the PutGet *law:*

$$put\ s\ v \mapsto s' \quad \text{implies} \quad get\ s' \mapsto v \qquad \text{for all } s,\ s' : S \text{ and } v : V$$

and the GetPut *law:*

$$get\ s \mapsto v \quad \text{implies} \quad put\ s\ v \mapsto s \qquad \text{for all } s : S \text{ and } v : V.$$

---

junction). The symbol '×' is also overloaded to denote *dependent pair types* — the inhabitants of a dependent pair type $(x : A) \times B\ x$ are pairs where the type of the second component depends on the value of the first component. Logically, dependent pair types are read as existentially quantified propositions — $(x : A) \times B\ x$ is read as "there exists $x : A$ such that the property B holds for x".

An underscore in a right-hand side expression instructs Agda to infer what should be placed there.

In Agda *the above becomes just a record definition:*[6]

```
record Lens (S V : Set) : Set₁ where
  field
    put : S → V → Par S
    get : S → Par V
    PutGet : {s s' : S} {v : V} → (put s v ↦ s') → (get s' ↦ v)
    GetPut : {s : S} {v : V} →    (get s ↦ v) → (put s v ↦ s)
```

Informally: Execution of *put* *s* *v* should produce an updated version of the source *s* by properly embedding all information of the view *v* into *s*. The *PutGet* law ensures that *put* does its job properly by requiring that, after *put* updates a source with a view, *get* can completely recover the view from the updated source. The *GetPut* law, on the other hand, ensures that *put* does nothing more than its designated job by requiring that *put* performs no update on a source if the view it receives is directly extracted from the source by *get*. Note that our definition of lenses is stronger than the one given by Foster et al [1]: our *PutGet* law says that successful computation of *put* guarantees success of the subsequent computation of *get*, whereas in Foster et al's definition there is no such guarantee; the *GetPut* law is similar.

A related notion is "partial" isomorphisms as defined by

```
record Iso (A B : Set) : Set₁ where
  field
    to   : A → Par B
    from : B → Par A
    to-from-inverse : {x : A} {y : B} → (to x ↦ y) → (from y ↦ x)
    from-to-inverse : {x : A} {y : B} → (from y ↦ x) → (to x ↦ y)
```

Lens is a generalisation of Iso because we can easily convert Iso *A* *B* to Lens *A* *B*, whose *put* directly produces a new source from its view argument, ignoring its source argument:

```
iso-lens : {A B : Set} → Iso A B → Lens A B
iso-lens iso = record
  { put = λ a b → Iso.from iso b
  ; get = Iso.to iso
  ; PutGet = Iso.from-to-inverse iso
  ; GetPut = Iso.to-from-inverse iso }
```

This conversion gives us a first way to produce lenses. For example, the identity isomorphism between the same type — whose *to* and *from* are both return —

[6] An Agda record is defined by listing the names and types of all its components, and a component type can depend on previous components. To access a component of an inhabitant of a record type, we should use the corresponding component extraction function. For example, Lens.*get* : {*S* *V* : Set} → Lens *S* *V* → *S* → Par *V* is for extracting the *get* component of a Lens.

gives rise to the "replacing" lens, and the empty isomorphism between any two types — whose *to* and *from* are both $\lambda\ \_ \to \mathsf{fail}$ — gives rise to the "failure" lens. Both lenses will appear in Section 4.

To give the reader a taste of how well-behavedness proofs are constructed in our Agda setting, let us try to define an operator $\_\leftrightarrow\_$ for lens composition:

$$
\begin{array}{l}
\_\leftrightarrow\_ : \{A\ B\ C : \mathsf{Set}\} \to \mathsf{Lens}\ A\ B \to \mathsf{Lens}\ B\ C \to \mathsf{Lens}\ A\ C \\
l \leftrightarrow r\ =\ \mathbf{record} \\
\quad \{\ \mathit{put}\ =\ \lambda\ a\ c \to \mathsf{Lens}.\mathit{get}\ l\ a\ \gg\!=\ \lambda\ b \to \mathsf{Lens}.\mathit{put}\ r\ b\ c\ \gg\!=\ \mathsf{Lens}.\mathit{put}\ l\ a \\
\quad ;\ \mathit{get}\ =\ \lambda\ a \to \mathsf{Lens}.\mathit{get}\ l\ a\ \gg\!=\ \mathsf{Lens}.\mathit{get}\ r \\
\quad ;\ \mathit{PutGet}\ =\ ? \\
\quad ;\ \mathit{GetPut}\ =\ ?\ \}
\end{array}
$$

The *put* and *get* functions for a composite lens can be pretty straightforwardly constructed from the *put* and *get* functions of the component lenses. As part of the definition of $\_\leftrightarrow\_$, we also need to prove that *PutGet* and *GetPut* hold for the pair of *put* and *get* we provide. For *PutGet* we need to show that, for all a, $a' : A$, and $c : C$, from a proof of $\mathit{put}\ a\ c\ \mapsto\ a'$ we can construct a proof of $\mathit{get}\ a'\ \mapsto\ c$. That is, we need to write a function converting an inhabitant of the type $\mathit{put}\ a\ c\ \mapsto\ a'$ to one of the type $\mathit{get}\ a'\ \mapsto\ c$. Agda automatically expands the two types following the definition of $\_\mapsto\_$:

$$
\begin{array}{ll}
 & \mathit{put}\ a\ c\ \mapsto\ a' \\
= & (\mathsf{Lens}.\mathit{get}\ l\ a\ \gg\!=\ \lambda\ b \to \mathsf{Lens}.\mathit{put}\ r\ b\ c\ \gg\!=\ \mathsf{Lens}.\mathit{put}\ l\ a)\ \mapsto\ a' \\
= & (b : B) \times (\mathsf{Lens}.\mathit{get}\ l\ a\ \mapsto\ b) \times \\
 & \qquad ((\mathsf{Lens}.\mathit{put}\ r\ b\ c\ \gg\!=\ \mathsf{Lens}.\mathit{put}\ l\ a)\ \mapsto\ a') \\
= & (b : B) \times (\mathsf{Lens}.\mathit{get}\ l\ a\ \mapsto\ b) \times \\
 & \qquad (b' : B) \times (\mathsf{Lens}.\mathit{put}\ r\ b\ c\ \mapsto\ b') \times (\mathsf{Lens}.\mathit{put}\ l\ a\ b'\ \mapsto\ a')
\end{array}
$$

and

$$
\begin{array}{ll}
 & \mathit{get}\ a'\ \mapsto\ c \\
= & (\mathsf{Lens}.\mathit{get}\ l\ a'\ \gg\!=\ \mathsf{Lens}.\mathit{get}\ r)\ \mapsto\ c \\
= & (b : B) \times (\mathsf{Lens}.\mathit{get}\ l\ a'\ \mapsto\ b) \times (\mathsf{Lens}.\mathit{get}\ r\ b\ \mapsto\ c)
\end{array}
$$

Thus all we need to do is convert a five-tuple to a three-tuple of the above types. This is easy: Applying $\mathsf{Lens}.\mathit{PutGet}\ l$ to the fifth component of type $\mathsf{Lens}.\mathit{put}\ l\ a\ b'\ \mapsto\ a'$ we obtain an inhabitant of type $\mathsf{Lens}.\mathit{get}\ l\ a'\ b'$, and applying $\mathsf{Lens}.\mathit{PutGet}\ r$ to the fourth component of type $\mathsf{Lens}.\mathit{put}\ r\ b\ c\ \mapsto\ b'$ we obtain an inhabitant of type $\mathsf{Lens}.\mathit{get}\ r\ b'\ c$. Hence the proof term for *PutGet* is simply

$$\lambda\ (b\ ,\ x\ ,\ b'\ ,\ y\ ,\ z) \to (b'\ ,\ \mathsf{Lens}.\mathit{PutGet}\ l\ z\ ,\ \mathsf{Lens}.\mathit{PutGet}\ r\ y)$$

With a similar reasoning, we can also construct a proof term for *GetPut*:

$$\lambda\ (b\ ,\ x\ ,\ y) \to (b\ ,\ x\ ,\ b\ ,\ \mathsf{Lens}.\mathit{GetPut}\ r\ y\ ,\ \mathsf{Lens}.\mathit{GetPut}\ l\ x)$$

completing the definition of $\_\leftrightarrow\_$. For the proofs in the rest of this paper we will merely provide a sketch, but these proofs all have their formal counterparts in Agda as illustrated above, completely guaranteeing their validity.

Various bidirectional programming languages have been built based on the idea of lenses. Such languages would contain a set of primitive lenses, like those we build by *iso-lens*, and a set of combinators which combine smaller lenses into larger ones, like what $\_\leftrightarrow\_$ does. Usually these languages are designed to suggest that what they describe are *get* programs, but it has recently been proposed that the opposite *putback-based* approach should be adopted instead, due to the following theorem:

Theorem 1 (putback is the essence). *Define, for any* mx, $my : \mathsf{Par}\ A$, *the type* $mx \approx my$ *to mean that* $mx \mapsto z$ *if and only if* $my \mapsto z$ *for any* $z : A$. *Then for any* S, $V : \mathsf{Set}$ *and two lenses* l, $r : \mathsf{Lens}\ S\ V$ *such that*

$$\mathsf{Lens}.put\ l\ s\ v \approx \mathsf{Lens}.put\ r\ s\ v \qquad \text{for all } s : S \text{ and } v : V,$$

we have

$$\mathsf{Lens}.get\ l\ s \approx \mathsf{Lens}.get\ r\ s \qquad \text{for all } s : S.$$

The theorem tells us that, when designing a bidirectional combinator, if we can justify the design of its *put* component, then the design of its *get* component needs no further justification, since there is no other *get* that can be paired with the same *put*. This is in sharp contrast with the *get*-based approach, with which the designer needs to choose for each *get* one or a few possible *put* strategies to be paired with the *get*. This complicates the design as there would be multiple combinators with the same *get* semantics, distinguished only by differences in their *put* semantics — this is rather counterintuitive since the programmer is supposed to think in terms of *get* instead of *put* when programming in *get*-based languages. Also the design is harder to justify as to why the set of particular *put* strategies are chosen, and it may well happen that, after the programmer describes a *get*, the corresponding *put* synthesised by the language is not what the programmer wants. All these problems simply do not exist for the putback-based approach.

4 BiGUL and its lens semantics

5 Reduction of common features

6 Discussion

Haskell implementation

Towards a dependently typed version

Theory construction in terms of AGDA programming
Regarding formalisation: "[W]e are interested in extending what can be calculated precisely because we are not interested in the calculations themselves. Developing techniques that allow more of a derivation to be calculated allows attention to be focussed on the creative parts, which cannot be calculated." [2]
[6]

References

1. J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3):17, 2007.
2. Jeremy Gibbons. An introduction to the Bird-Meertens formalism. In *New Zealand Formal Program Development Colloquium*, pages 1–12, 1994.
3. Jeremy Gibbons. Calculating functional programs. In *Algebraic and Coalgebraic Methods in the Mathematics of Program Construction*, number 2297 in Lecture Notes in Computer Science, chapter 5, pages 149–201. Springer-Verlag, 2002.
4. Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.
5. Hugo Pacheco, Zhenjiang Hu, and Sebastian Fischer. Monadic combinators for "putback" style bidirectional programming. In *Partial Evaluation and Program Manipulation*, PEPM'14, pages 39–50. ACM, 2014.
6. The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, Princeton, 2013.
7. Philip Wadler. The essence of functional programming. In *Principles of Programming Languages*, POPL'92, pages 1–14. ACM, 1992.